

运维巡检 AI Agent 设计文档

Context

公司运维团队在事件发生时，需要人工逐一巡查 Prometheus、Zabbix、ELK、云监控等多套监控系统，效率低且容易遗漏。本项目旨在构建一个 AI Agent，接收自然语言事件描述，自动并行扫描所有监控系统，输出结构化的巡检报告（自然语言），发送至事件处置群，替代人工巡查流程。

不做根因定位，只负责发现"哪里有异常"。

技术选型

项	选择
语言	Python 3.11+
Web 框架	FastAPI (全异步)
LLM	可插拔 (OpenAI / Claude / 国产模型，通过抽象层切换)
HTTP 客户端	httpx (async)
配置	Pydantic Settings + YAML
日志	structlog (结构化日志)

核心架构：三阶段混合扫描



项目结构

```

operation-agent/
├── app/
│   ├── main.py           # FastAPI 入口 · lifespan 管理
│   ├── config.py         # 配置管理
│   └── api/
│       ├── v1/
│       │   ├── router.py  # 路由聚合
│       │   ├── incidents.py # POST /incidents 触发巡检
│       │   └── health.py   # GET /health
│       ├── api/schemas/
│       │   ├── incident.py # 请求/响应模型
│       │   └── monitoring.py # AlertItem, MetricSnapshot, LogEntry
│       └── agent/
│           ├── orchestrator.py # 三阶段编排引擎 (核心)
│           ├── baseline_scanner.py # Phase 1: 并行基础扫描
│           ├── deep_investigator.py # Phase 2: LLM 驱动深入调查
│           ├── report_synthesizer.py # Phase 3: 报告生成
│           └── tool_registry.py # LLM 可调用的工具定义
│       ├── adapters/
│       │   ├── base.py      # MonitoringAdapter ABC + Registry
│       │   ├── prometheus.py # Prometheus + Grafana
│       │   ├── zabbix.py    # Zabbix
│       │   ├── elasticsearch.py # ELK / OpenSearch
│       │   ├── cloudwatch.py # AWS CloudWatch
│       │   └── alibaba_cloud.py # 阿里云 CloudMonitor
│       ├── llm/
│       │   ├── base.py      # LLMPProvider ABC
│       │   ├── openai_provider.py # OpenAI (兼容 OpenAI 接口的模型也用这个)
│       │   ├── anthropic_provider.py
│       │   └── factory.py    # 工厂 · 根据配置创建 provider
│       ├── notify/
│       │   ├── base.py      # Notifier ABC
│       │   └── webhook.py    # 钉钉/飞书/Slack 等 webhook 通知
│       ├── core/
│       │   ├── exceptions.py # 异常体系
│       │   └── dependencies.py # FastAPI 依赖注入
│       └── prompts/
│           ├── deep_investigation.py # Phase 2 提示词模板
│           └── report_synthesis.py   # Phase 3 提示词模板
├── config/
│   └── config.example.yaml # 配置示例
├── tests/
├── pyproject.toml
├── Dockerfile
└── .env.example

```

关键接口设计

1. 监控适配器接口 (Adapter)

每个监控系统实现 `MonitoringAdapter` 抽象类：

```

from abc import ABC, abstractmethod
from enum import Flag, auto
from dataclasses import dataclass

class Capability(Flag):
    ALERTS = auto()
    METRICS = auto()
    LOGS = auto()

@dataclass
class AlertItem:
    source: str          # adapter name
    severity: str        # critical / warning / info
    title: str
    description: str
    started_at: datetime
    labels: dict[str, str]

@dataclass
class MetricSnapshot:
    source: str
    metric_name: str
    values: list[tuple[datetime, float]]
    labels: dict[str, str]

@dataclass
class LogEntry:
    source: str
    timestamp: datetime
    level: str
    message: str
    fields: dict[str, Any]

class MonitoringAdapter(ABC):
    @property
    @abstractmethod
    def name(self) -> str: ...

    @property
    @abstractmethod
    def capabilities(self) -> Capability: ...

    @abstractmethod
    async def get_active_alerts(self) -> list[AlertItem]:
        """Phase 1: 获取当前活跃告警"""
        ...

```

```

async def query_metrics(
    self, query: str, start: datetime, end: datetime
) -> list[MetricSnapshot]:
    """Phase 2: 执行自定义指标查询 ( 可选 ) """
    raise NotImplementedError

async def search_logs(
    self, query: str, start: datetime, end: datetime, limit: int = 100
) -> list[LogEntry]:
    """Phase 2: 搜索日志 ( 可选 ) """
    raise NotImplementedError

```

通过 `AdapterRegistry` 统一管理：

```

class AdapterRegistry:
    def __init__(self):
        self._adapters: dict[str, MonitoringAdapter] = {}

    def register(self, adapter: MonitoringAdapter) -> None:
        self._adapters[adapter.name] = adapter

    def get(self, name: str) -> MonitoringAdapter:
        return self._adapters[name]

    def all(self) -> list[MonitoringAdapter]:
        return list(self._adapters.values())

    def with_capability(self, cap: Capability) -> list[MonitoringAdapter]:
        return [a for a in self._adapters.values() if cap in a.capabilities]

```

新增监控系统只需：

1. 实现 adapter 类
2. 加配置
3. 注册到 registry

不需要修改编排器或其他模块。

2. LLM 抽象层

```

@dataclass
class ToolDefinition:
    name: str
    description: str
    parameters: dict          # JSON Schema
    handler: Callable         # 实际执行函数

```

```

@dataclass
class LLMMessage:
    role: str          # system / user / assistant / tool
    content: str
    tool_calls: list | None = None
    tool_call_id: str | None = None

@dataclass
class LLMResponse:
    content: str
    tool_calls: list[ToolCall] | None = None
    usage: dict | None = None

class LLMProvider(ABC):
    @abstractmethod
    async def chat(self, messages: list[LLMMessage]) -> LLMResponse:
        """普通对话 ( Phase 3 报告生成 ) """
        ...

    @abstractmethod
    async def chat_with_tools(
        self, messages: list[LLMMessage], tools: list[ToolDefinition]
    ) -> LLMResponse:
        """带 tool calling 的对话 ( Phase 2 深入调查 ) """
        ...

```

各 provider 内部负责转换为自己的 API 格式。OpenAI provider 支持 `base_url`，可兼容所有 OpenAI 兼容接口。

3. API 接口

```

POST /api/v1/incidents
{
  "description": "用户反馈 XX 业务访问超时",
  "time_window_minutes": 60,
  "adapters": null,           // null = 扫描所有
  "notify": true
}

→ 返回:
{
  "incident_id": "uuid",
  "status": "completed",
  "report": "巡检报告自然语言文本...",
  "phases": {
    "baseline": { "alerts_count": 12, "adapters_scanned": 3, "errors": [] },
    "investigation": { "rounds": 2, "queries_executed": 5 },
    "synthesis": { "generated": true }
  },
}

```

```
"duration_seconds": 45.2
}
```

V1 同步返回（典型耗时 30-90s）· 后续可扩展为异步。

三阶段编排详细设计

Phase 1: 基础巡检 (Baseline Scanner)

```
async def scan(self, adapters: list[MonitoringAdapter]) -> BaselineScanResult:
    """并行查询所有 adapter 的活跃告警"""
    tasks = [
        asyncio.wait_for(adapter.get_active_alerts(), timeout=15.0)
        for adapter in adapters
    ]
    results = await asyncio.gather(*tasks, return_exceptions=True)

    alerts, errors = [], []
    for adapter, result in zip(adapters, results):
        if isinstance(result, Exception):
            errors.append(AdapterError(adapter.name, str(result)))
        else:
            alerts.extend(result)

    return BaselineScanResult(alerts=alerts, errors=errors)
```

Phase 2: 深入调查 (Deep Investigator)

LLM 通过 tool calling 决定执行哪些查询：

可用 Tools:

- | | |
|---|---------------------|
| - query_metrics(adapter, query, start, end) | → 查指标 |
| - search_logs(adapter, query, start, end) | → 查日志 |
| - get_adapter_info() | → 列出可用 adapter 及其能力 |
| - finish_investigation(findings) | → 结束调查 · 输出发现 |

循环流程：

1. 将事件描述 + Phase 1 结果发给 LLM
2. LLM 返回 tool calls
3. 执行 tool calls · 将结果返回 LLM
4. 重复 · 最多 3 轮
5. LLM 调用 finish_investigation 或达到轮次上限时结束

Phase 3: 报告生成 (Report Synthesizer)

将所有上下文交给 LLM · 生成结构化自然语言报告：

输入：

- 事件描述
- Phase 1 告警列表
- Phase 2 深入调查发现
- 不可达的系统列表

输出：

- 巡检报告（自然语言，Markdown 格式）
- 包含：概览、各系统异常详情、数据缺失说明

降级策略：LLM 不可用时，使用模板引擎生成结构化报告（列出告警，无分析）。

配置结构

```
# config/config.example.yaml

app:
  name: "operation-agent"
  debug: false
  timeout_total: 120          # 整体超时（秒）
  timeout_adapter: 15         # 单 adapter 超时（秒）
  timeout_phase2: 60          # Phase 2 累计超时（秒）
  phase2_max_rounds: 3        # Phase 2 最大迭代轮次

llm:
  provider: "openai"          # openai / anthropic
  model: "gpt-4o"
  api_key: "${LLM_API_KEY}"
  base_url: null              # 自定义 base_url（兼容国产模型）
  temperature: 0.1

adapters:
  prometheus:
    enabled: true
    base_url: "http://prometheus:9090"
    timeout: 10

  zabbix:
    enabled: false
    base_url: "http://zabbix/api_jsonrpc.php"
    username: "${ZABBIX_USER}"
    password: "${ZABBIX_PASS}"

  elasticsearch:
    enabled: false
    hosts:
      - "http://es-node1:9200"
    index_pattern: "app-logs-*"
    username: "${ES_USER}"
```

```
password: "${ES_PASS}"

cloudwatch:
  enabled: false
  region: "us-east-1"
  aws_access_key_id: "${AWS_ACCESS_KEY}"
  aws_secret_access_key: "${AWS_SECRET_KEY}"

alibaba_cloud:
  enabled: false
  region: "cn-hangzhou"
  access_key_id: "${ALIYUN_AK}"
  access_key_secret: "${ALIYUN_SK}"

notify:
  enabled: true
  webhooks:
    - name: "dingtalk-ops"
      url: "${DINGTALK_WEBHOOK_URL}"
      secret: "${DINGTALK_WEBHOOK_SECRET}"
      type: "dingtalk"
    - name: "feishu-ops"
      url: "${FEISHU_WEBHOOK_URL}"
      type: "feishu"

logging:
  level: "INFO"
  format: "json"                # json / console
```

容错策略

场景	处理方式
单个 adapter 查询 超时/失败	Phase 1 用 <code>asyncio.gather(return_exceptions=True)</code> 并行查询，单个失败不 阻塞，报告中标注该系统不可达
LLM 不可用	跳过 Phase 2，Phase 3 降级为模板生成（列出告警，无分析）
Phase 2 某次 tool call 失败	将错误信息返回给 LLM，由 LLM 决定是否重试或跳过
整体超时（120s）	强制终止，用已收集的数据生成部分报告
通知发送失败	记录日志，不影响报告返回
配置中 adapter 未 启用	跳过，不报错

异常体系


```

class AgentBaseError(Exception):
    """所有自定义异常的基类"""

class AdapterError(AgentBaseError):
    """适配器相关错误"""
    def __init__(self, adapter_name: str, message: str): ...

class AdapterTimeoutError(AdapterError):
    """适配器查询超时"""

class AdapterConnectionError(AdapterError):
    """适配器连接失败"""

class LLMError(AgentBaseError):
    """LLM 相关错误"""

class LLMLimitError(LLMError):
    """LLM 限流"""

class OrchestrationError(AgentBaseError):
    """编排流程错误"""

class NotifyError(AgentBaseError):
    """通知发送错误"""

```

Webhook 通知设计

支持钉钉、飞书、Slack 等 webhook 格式：

```

class Notifier(ABC):
    @abstractmethod
    async def send(self, title: str, content: str) -> None: ...

class WebhookNotifier(Notifier):
    """通用 webhook 通知器"""

    async def send(self, title: str, content: str) -> None:
        for webhook in self.webhooks:
            payload = self._format_payload(webhook.type, title, content)
            await self._client.post(webhook.url, json=payload)

    def _format_payload(self, type: str, title: str, content: str) -> dict:
        match type:
            case "dingtalk":
                return {
                    "msgtype": "markdown",
                    "markdown": {"title": title, "text": content}
                }

```

```

    case "feishu":
        return {
            "msg_type": "interactive",
            "card": { ... }
        }
    case "slack":
        return {
            "text": title,
            "blocks": [ ... ]
        }

```

Tool Registry 设计

Phase 2 中 LLM 可调用的工具：

Tool 名称	描述	参数
query_metrics	查询指定 adapter 的指标数据	adapter, query, start, end
search_logs	搜索指定 adapter 的日志	adapter, query, start, end, limit
get_adapter_info	列出所有可用 adapter 及其能力	无
finish_investigation	结束调查 · 输出发现列表	findings: list[str]

工具注册机制：

```

class ToolRegistry:
    def __init__(self, adapter_registry: AdapterRegistry):
        self._tools: dict[str, ToolDefinition] = {}
        self._register_builtin_tools(adapter_registry)

    def _register_builtin_tools(self, registry: AdapterRegistry):
        self.register(ToolDefinition(
            name="query_metrics",
            description="查询指定监控系统的指标数据",
            parameters={
                "type": "object",
                "properties": {
                    "adapter": {"type": "string", "description": "适配器名称"},
                    "query": {"type": "string", "description": "查询表达式"},
                    "start": {"type": "string", "description": "开始时间
ISO8601"},
                    "end": {"type": "string", "description": "结束时间 ISO8601"},
                },
                "required": ["adapter", "query", "start", "end"]
            },
            handler=self._handle_query_metrics
        ))
        # ... 同理注册其他工具

```

```
def get_tool_definitions(self) -> list[ToolDefinition]:  
    return list(self._tools.values())  
  
async def execute(self, name: str, arguments: dict) -> str:  
    tool = self._tools[name]  
    return await tool.handler(**arguments)
```

Prompt 设计

Phase 2: 深入调查提示词

你是一个运维巡检助手。当前发生了一个事件，你需要根据事件描述和已有的基础巡检结果，决定是否需要进行进一步查询指标或日志来补充信息。

事件描述

{incident_description}

基础巡检结果（活跃告警）

{baseline_alerts}

可用工具

你可以使用以下工具进行深入调查：

- query_metrics: 查询指标数据
- search_logs: 搜索日志
- get_adapter_info: 查看可用的监控系统
- finish_investigation: 当你认为信息足够时，调用此工具结束调查

要求

1. 根据事件描述，判断需要查询哪些相关指标和日志
2. 每轮最多调用 3 个工具
3. 如果基础巡检已经提供了足够信息，直接调用 finish_investigation
4. 不要做根因分析，只收集异常信息

Phase 3: 报告生成提示词

你是一个运维巡检报告撰写助手。请根据以下信息，生成一份结构化的巡检报告。

事件描述

{incident_description}

巡检发现

{all_findings}

不可达系统

{unreachable_systems}

报告要求

1. 使用 Markdown 格式

2. 包含：概览摘要、各系统异常详情、数据缺失说明
 3. 按严重程度排列异常
 4. 语言简洁，面向运维工程师
 5. 不做根因推测，只陈述发现的异常事实

实现顺序

阶段	内容	产出
1	项目骨架：main.py、配置、health 端点、pyproject.toml	可启动的空服务
2	Adapter 接口 + Registry + Prometheus adapter	Phase 1 数据源就绪
3	Phase 1 基础扫描 + 编排器	能并行获取告警
4	LLM 抽象层 + OpenAI provider	LLM 调用就绪
5	Phase 2 深入调查 + tool registry	端到端 Phase 1+2
6	Phase 3 报告生成，打通端到端流程	完整三阶段流程
7	通知模块 (webhook)	报告可发送
8	更多 adapter：Zabbix → Elasticsearch → 云监控	全监控系统覆盖
9	测试	质量保障

验证方式

1. 启动验证：GET /health 确认 adapter 连接状态
 2. 端到端验证：POST /api/v1/incidents 传入测试事件，验证三阶段流程
 3. 容错验证：模拟 adapter 不可用，验证降级和容错
 4. 单元测试：mock adapter 和 LLM，测试编排逻辑

未来扩展 (不在本期范围)

- 异步任务模式 (WebSocket / SSE 推送进度)
 - 巡检历史存储与回溯
 - 自定义巡检 playbook
 - 与 ITSM 系统集成
 - 多租户支持